JSS MAHAVIDYAPEETHA, MYSURU
JSS ACADEMY OF TECHNICAL EDUCATION, BENGALURU-560 060

DEPARTMENT OF CSE

Report on

"EchoLang: Design and Implementation of a Modular Software Architecture for Multilingual Speech Processing on Resource-Constrained Hardware"

Submitted By
Shubham Pandey(1JS21CS140)

For the Subject

Final Year Project – 21CSP76

Department of CSE,
JSSATE- Bengaluruu

Abstract

EchoLang is the design, development, and evaluation of a modular software framework called EchoLang, which combines three key features of multilingual speech processing: speech recognition, machine translation, and speech synthesis. It is a software framework that works with English, Hindi, and Kannada languages, and is specifically designed to work only on consumer-level hardware, i.e., CPUs, which makes certain demands in terms of memory management and model loading strategy.

The main contribution is a layered software architecture that separates pipeline orchestration from individual model backends. This makes it possible to put together different AI components behind the same interface. The system adds language-specific model selection logic to FasterWhisper (CTranslate2 runtime) at the STT layer. The translation layer hides AI4Bharat's IndicTrans2 models and lets you use six different language pairs through one API. The TTS layer uses a strategy pattern to switch between two very different synthesis backends: Coqui XTTS-v2 for English and Hindi and Facebook MMS-TTS for Kannada. The choice is based on the quality of the output and the language's runtime compatibility. This routing architecture lets you replace or upgrade individual parts without changing the logic of the pipeline.

The system solves a number of real-world software engineering problems, such as how to initialize large neural network models without using more than 8 GB of memory, how to change Whisper models to int8-quantized CTranslate2 format to save space on both disk and memory, and how to manage model paths and language-to-backend mapping based on configuration. The application has a Gradio-based web interface with tabs that lets users interact with it and test each stage of the pipeline in a structured way.

Evaluation includes checking for functional correctness in all supported language pairs and processing modes, profiling runtime performance under CPU-only constraints, and finding bottlenecks at the component level. The project shows that careful architectural decomposition and resource-aware design can make it possible for complex multi-model AI applications to run on cheap hardware. It also shows the trade-offs that come with integrating open-source ML components that were made by different people.

Full project repository: https://github.com/shoibum/EchoLang

Chapter 1: Introduction

1.1 Overview

Multilingual speech processing — the ability to transcribe, translate, and synthesize speech across languages — has seen significant advances with large-scale deep learning models. Systems such as OpenAI's Whisper, Meta's MMS, and various neural machine translation frameworks have demonstrated impressive capabilities across dozens of languages. However, deploying these models in practice presents a fundamental systems engineering challenge: most state-of-the-art models are designed for GPU-accelerated environments with substantial memory and compute resources.

This constraint is particularly acute for low-resource Indian languages such as Hindi and Kannada, where communities that would benefit most from speech technology often have the least access to high-end hardware. The problem is not merely one of model accuracy — it is a systems engineering problem: how to compose multiple AI components into a coherent, usable pipeline that operates within the constraints of consumer-grade hardware.

1.2 Objectives of the Research

- Design a modular pipeline architecture composing speech-to-text (STT), machine translation (MT), and text-to-speech (TTS) subsystems into a unified system supporting English, Hindi, and Kannada.
- Implement language-aware routing logic that dispatches inputs to the appropriate model backend at each pipeline stage, accommodating heterogeneity of available open-source models.
- Optimize the system for CPU-only deployment on consumer hardware (8 GB RAM) using model quantization, lazy loading, and memory-conscious resource management.
- Develop a web-based user interface exposing five distinct processing workflows enabling systematic evaluation of pipeline performance at each composition depth.
- Evaluate the system's latency, memory footprint, and output quality to characterize the feasibility and limitations of fully local multilingual speech AI.
- Assess the social and practical impact of democratizing speech AI for low-resource Indian languages.

1.3 Scope of the Research

EchoLang targets three languages: English, Hindi, and Kannada. English serves as the high-resource baseline with mature model support across all subsystems. Hindi represents a medium-resource language with growing but uneven model coverage. Kannada represents a low-resource Dravidian language where model availability is limited and quality varies significantly across tasks.

The system is designed for local deployment on a single machine with no internet dependency after initial model downloads. Real-time streaming is not within the current scope; the system processes complete audio files or text inputs in batch mode. The scope also covers ethical considerations around privacy-preserving local AI and the broader social implications of inclusive speech technology for underrepresented linguistic communities.

Chapter 2: Literature Review

The following review covers the primary research underpinning EchoLang's three subsystems: automatic speech recognition, neural machine translation for Indian languages, and multilingual text-to-speech synthesis. Each area has seen rapid advances driven by transformer-based architectures and large-scale multilingual pretraining.

Paper / System	Methodology	Key Findings
Whisper (Radford et al., 2022)	Transformer-based encoder-decoder trained on 680K hours of weakly supervised multilingual audio data.	Achieves near-human WER on English and competitive performance across 97 languages. Demonstrates that scale and weak supervision can substitute for curated datasets.
faster-whisper / CTranslate2 (SYSTRAN, 2023)	CTranslate2 reimplementation of Whisper with int8 quantization and CPU-optimized inference kernels.	Provides 3–5x speedup and 4x memory reduction over PyTorch Whisper on CPU, making interactive transcription feasible without GPU. Used directly as EchoLang's STT backend.
vasista22 Fine-tuned Whisper (2023)	Fine-tuning of Whisper Small on curated Hindi and Kannada speech corpora.	Significantly reduces WER for Hindi and Kannada compared to the base multilingual Whisper model. Demonstrates value of language-specific fine-tuning for low-resource Indic languages.
IndicTrans2 (Gala et al., 2023)	Transformer-based NMT system trained on large-scale parallel corpora for all 22 scheduled Indian languages, with distilled 200M-parameter variants.	Achieves state-of-the-art BLEU scores for English-Indic and Indic-Indic translation. Distilled models retain most quality while fitting within memory constraints of consumer hardware.
XTTS-v2 (Coqui AI, 2023)	Cross-lingual TTS model with voice cloning capability, supporting 17 languages including Hindi. Based on VITS architecture with cross-lingual conditioning.	Produces near-natural speech for supported languages on CPU. Voice cloning from short reference audio enables consistent speaker identity across sessions.
MMS-TTS (Pratap et al., 2023)	Facebook's Massively Multilingual Speech system covering 1,000+ languages including Kannada, using VITS architecture trained on diverse multilingual data.	Provides the only viable open-source TTS option for Kannada on CPU. Output is intelligible but shows reduced naturalness compared to XTTS-v2 for better-resourced languages.

Paper / System	Methodology	Key Findings
IndicTransToolkit (Gumma, 2023)	Pre/postprocessing pipeline for IndicTrans2 handling script normalization, tokenization, and transliteration for Indic languages.	Essential for correctly handling Devanagari (Hindi) and Kannada scripts. Manages language code mapping between IndicTrans2 conventions and standard ISO codes.

Table 2.1: Literature Review Summary — Core Technologies Underpinning EchoLang

The literature establishes that while individual components (ASR, MT, TTS) have matured sufficiently for local deployment, no prior work has integrated these heterogeneous systems into a single unified pipeline optimized for resource-constrained hardware. EchoLang addresses this gap.

Chapter 3: Problem Definition

India's linguistic diversity — 22 scheduled languages and hundreds of dialects — creates a pressing need for multilingual speech tools that go beyond English. Despite significant advances in speech AI, most practical systems remain inaccessible to users in low-resource language communities for two compounding reasons:

- State-of-the-art speech models require GPU infrastructure (typically 8–24 GB VRAM), which is economically inaccessible for individual users and small organizations in rural or semi-urban India.
- Existing tools for Indian languages are fragmented across different research groups, model families, and APIs, with no unified system offering end-to-end speech translation for languages like Kannada.

The core problem EchoLang addresses is therefore a systems integration and resource optimization problem: how to compose the best available open-source models for STT, MT, and TTS into a coherent multilingual speech pipeline that operates on a standard consumer laptop (8 GB RAM, CPU only) with no cloud dependency.

Secondary problems include:

- Heterogeneity: the target models have different architectures, frameworks, input/output formats, and resource requirements. There is no common interface or integration layer.
- Memory pressure: loading all models simultaneously for a three-stage pipeline would exceed 8 GB RAM without careful quantization and lazy loading.
- Quality asymmetry: model quality varies significantly across languages, particularly for TTS in Dravidian languages. The system must degrade gracefully rather than fail when lower-quality components are used.
- Privacy: cloud-based speech APIs process audio on external servers, which is inappropriate for sensitive personal or organizational speech data. A fully local system eliminates this risk.

In addition to dataset creation and model training, this project aims to demonstrate that careful architectural decomposition can make complex multi-model AI applications accessible to users who have historically been excluded from the benefits of speech technology due to hardware constraints and language resource gaps.

Chapter 4: Methodology

4.1 System Architecture

EchoLang follows a layered architecture separating concerns across three tiers: the model layer, the pipeline orchestration layer, and the presentation layer.

Subsystem	Backend	Languages	Format
STT	FasterWhisper (CTranslate2)	English, Hindi, Kannada	int8 quantized
MT	IndicTrans2 (distilled 200M)	en↔hi, en↔kn, hi↔kn	PyTorch
TTS	XTTS-v2 / MMS-TTS	en, hi / kn	PyTorch

Table 4.1: Model backends used in EchoLang

The model layer contains wrapper classes for each AI backend, encapsulating model initialization, input preprocessing, inference, and output postprocessing behind consistent interfaces. The pipeline orchestration layer (pipeline.py) composes these wrappers into end-to-end workflows, handling language detection, routing, intermediate format conversions, and error propagation. The presentation layer uses Gradio to provide a tabbed web interface.

4.2 Pipeline Design

The pipeline supports five workflows of increasing composition depth:

- Speech to Text: Audio input → STT → transcribed text output.
- Text to Speech: Text input → TTS → synthesized audio output. Routes to XTTS-v2 or MMS-TTS based on target language.
- Text to Translated Text: Text input → MT → translated text output.
- Speech to Translated Text: Audio → STT → MT → translated text. Two-stage composition.
- Speech to Translated Speech: Audio → STT → MT → TTS → synthesized audio. Full three-stage pipeline.

4.3 Language-Aware Routing

A key design decision is the language-aware routing layer. For STT, three separate Whisper models are maintained (fine-tuned Hindi, fine-tuned Kannada, and base English), mapped by the user-selected language. For MT, routing determines which IndicTrans2 directional model to load (En→Indic, Indic→En, or Indic→Indic). For TTS, the binary routing decision is XTTS-v2 for English/Hindi and MMS-TTS for Kannada, based on empirical quality assessment.

4.4 Implementation Details

The STT subsystem uses FasterWhisper with int8 quantization, reducing model size by approximately 4x and providing 2–3x CPU inference speedup. The MT subsystem uses distilled 200M-parameter IndicTrans2 variants loaded on demand, with only the required directional model held in memory at any time. The TTS subsystem implements a strategy

pattern with XTTS-v2 for English/Hindi and MMS-TTS for Kannada, with output normalized to a consistent WAV format (22050 Hz, 16-bit PCM).

Resource management strategies include:

- Lazy model loading: no models are loaded at startup; each is initialized only when first needed.
- Int8 quantization: Whisper models are converted to CTranslate2 int8 format, reducing footprint from ~960 MB to ~240 MB per model.
- Distilled translation models: 200M-parameter variants used instead of 1.1B-parameter full models.
- Sequential pipeline execution: each stage completes and releases intermediate tensors before the next stage begins.

4.5 Evaluation Methodology

Experiments were conducted on a consumer-grade laptop (8 GB RAM, CPU only, no GPU). Test inputs consisted of short audio clips (5–15 seconds) in Hindi, English, and Kannada, plus typed text in all three languages. Each measurement was repeated three times and the median value reported. Qualitative evaluation was conducted with bilingual speakers of English-Hindi and English-Kannada.

Workflow	First Run (incl. load)	Subsequent Runs
Speech to Text (10s audio)	15–25 s	3–8 s
Text to Speech (1 sentence)	20–40 s	5–15 s
Text Translation	10–20 s	2–5 s
Speech to Translated Text	25–45 s	5–12 s
Speech to Translated Speech	40–75 s	10–25 s

Table 4.2: End-to-end latency by workflow (median of 3 runs)

Component	Peak RAM	Notes
Baseline (app only)	~200 MB	Python + Gradio + dependencies
+ STT (1 Whisper model)	~500 MB	Int8 quantized CTranslate2 model
+ MT (1 IndicTrans2 model)	~1.2 GB	Distilled 200M model + tokenizer
+ TTS (XTTS-v2)	~2.8 GB	Largest single component
Full pipeline peak	~4.5–6 GB	All models loaded simultaneously

Table 4.3: Approximate RAM usage by component

Chapter 5: Expected Outcomes

5.1 Technical Outcomes

- A fully functional, locally deployable multilingual speech pipeline supporting English, Hindi, and Kannada across five distinct processing workflows, operating within 8 GB RAM on consumer CPU hardware.
- Demonstrated feasibility of int8-quantized CPU inference for interactive speech applications, with measured latency and memory profiles characterizing the trade-offs involved.
- A reusable modular architecture where individual components (STT, MT, TTS backends) can be upgraded or replaced without modifying the pipeline orchestration or user interface layers.
- Empirical latency and memory benchmarks establishing baselines for future work on low-resource multilingual speech systems.

5.2 Research Contributions

- An integration framework demonstrating that heterogeneous open-source AI models (FasterWhisper, IndicTrans2, XTTS-v2, MMS-TTS) can be composed into a coherent system despite their architectural and framework differences.
- Identification of the quality gap between high-resource (English) and low-resource (Kannada) languages across all three pipeline stages, providing concrete evidence for where model development efforts should be directed.
- A practical reference implementation for CPU-only multilingual speech AI, available as open source, that researchers and developers building for Indian language communities can build upon.

5.3 Social Impact

- Providing a privacy-preserving, locally deployable speech tool that requires no cloud subscription, making multilingual speech AI accessible to users in low-bandwidth or data-sensitive environments.
- Demonstrating a viable path toward speech technology inclusion for Kannada-speaking communities, who are underserved by existing commercial speech APIs.
- Contributing to the broader ecosystem of AI tools for underrepresented Indian languages, encouraging further research in regional language speech processing.

5.4 Future Enhancements

- Replacing XTTS-v2 with a lighter TTS model (e.g., VITS or Piper TTS) to enable operation on 4 GB RAM systems.
- Adding automatic language detection at the STT stage to handle code-mixed and multilingual input.
- Integrating WebSocket-based streaming for real-time transcription and translation.
- Expanding language support to Tamil, Telugu, Bengali, and Marathi using the same modular architecture.

Chapter 6: Project Timeline

Total Duration: 4 Months

Phase	Duration	Tasks	Milestone
Phase 1: Literature Review & Architecture Design	Weeks 1–2	Review ASR, MT, and TTS literature. Evaluate candidate models for each subsystem. Define pipeline architecture and component interfaces.	Architecture design document and model selection finalized.
Phase 2: Model Integration & Backend Development	Weeks 3–7	Implement FasterWhisper STT wrapper with int8 quantization. Integrate IndicTrans2 MT with IndicTransToolkit. Implement XTTS-v2 and MMS-TTS TTS backends. Develop language-aware routing layer.	All three subsystems functional independently with unit tests passing.
Phase 3: Pipeline Orchestration & UI	Weeks 8–10	Implement pipeline.py composing subsystems into five workflows. Build Gradio web interface with tabbed layout. Implement lazy loading and memory management strategies.	Full end-to-end pipeline operational across all five workflows.
Phase 4: Evaluation & Optimization	Weeks 11–13	Conduct latency profiling across all workflows. Measure per-component RAM usage. Qualitative evaluation with bilingual speakers. Identify and address bottlenecks.	Evaluation results documented; system optimized within 8 GB RAM target.
Phase 5: Documentation & Submission	Weeks 14–16	Write project report. Document architecture, implementation decisions, and trade-offs. Prepare GitHub repository with README and setup instructions. Final submission.	Final project report submitted; public repository released.

Table 6.1: Project Timeline

Chapter 7: Conclusion

EchoLang demonstrates that a fully local, CPU-only multilingual speech processing system is feasible with current open-source models and careful systems engineering. The modular architecture successfully composes heterogeneous AI components — FasterWhisper for STT, IndicTrans2 for MT, and XTTS-v2/MMS-TTS for TTS — into a unified pipeline supporting English, Hindi, and Kannada across five distinct processing workflows.

The project highlights both the opportunities and limitations of local AI deployment for low-resource languages. On the opportunity side, quantization and model distillation have advanced to the point where interactive multilingual speech processing is achievable on 8 GB RAM with no GPU. On the limitation side, the quality gap between high-resource (English) and low-resource (Kannada) languages remains significant, particularly in TTS, and is unlikely to be closed by engineering alone — it requires better training data and models for these languages.

The modular design of EchoLang ensures that as individual components improve — better Kannada TTS models, more efficient translation architectures, faster CPU inference runtimes — they can be integrated without redesigning the system. This extensibility is the most valuable architectural contribution of the project.

References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," arXiv:2212.04356, 2022.

[2] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," GitHub, 2023.

[3] J. Gala, P. P. Jayakumar, et al., "IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages," Transactions of the ACL, 2023.

[4] V. Gumma, "IndicTransToolkit," GitHub, 2023.

[5] Coqui AI, "XTTS-v2: Cross-lingual Text-to-Speech," HuggingFace, 2023.

[6] V. Pratap, A. Tjandra, et al., "Scaling Speech Technology to 1,000+ Languages," arXiv:2305.13516, 2023.

[7] vasista22, "whisper-hindi-small and whisper-kannada-small," HuggingFace, 2023.

[8] CTranslate2 documentation, "Quantization," Available: https://opennmt.net/CTranslate2/quantization.html

[9] Gradio, "Gradio: Build Machine Learning Web Apps," Available: https://gradio.app

[10] AI4Bharat, "IndicTrans2," GitHub, 2023. Available: https://github.com/AI4Bharat/IndicTrans2